



Prof. Dr. Steffen Wagner
Angewandte Statistik
Fachbereich II
Berliner Hochschule für Technik

Exercise 08: Neural Networks - the basics

Machine Learning II – WiSe 25

1 Preparations	2
1.1 Required Packages	2
2 Remarks (Must read!)	2
2.1 Data	2
2.2 Tips	2
2.3 Functions	3
2.4 Optimization function	3
3 Exercises	4
3.1 One hidden layer with 1 node	4
3.2 One hidden layer with 2 nodes	4
3.3 Neural network as a classifier: one hidden layer with 2 nodes, output layer with 3 nodes	5
3.4 One-dimensional optimisation	6
3.5 Two-dimensional optimisation	6
3.6 Linear model parameter optimisation	7
4 Written Homework exercises	8
4.1 Newton-Raphson method: finding a root of a function	8
4.2 Finding a minimum of a function	8
4.3 Chain rule in back propagation	9



1 Preparations

As in Machine Learning 1 you can either use R and RStudio on the lab computers or your own computer.

1.1 Required Packages

For this exercise no additional R packages are required.

2 Remarks (Must read!)

In this workshop you will code a basic neural network from scratch, using the basic definitions of a NN. The purpose of the workshop is to learn the fundamental principles of a NN rather than writing code for proper data analysis.

The following material is available via Moodle and has to be downloaded first:

- R scripts:
 - ML2_11_NN_I_1.R
 - ML2_11_NN_I_2.R
 - ML2_11_NN_I_3.R
 - ML2_11_optim_1d.R
 - ML2_11_optim_2d.R
 - ML2_11_optim_ols.R
- Data sets:
 - NN_I_regr_data.Rda
 - NN_I_class_data.Rda

Save the data files in a sub-folder of your RStudio Project named Data.

Important: Please read all of this worksheet before working through the script file.

2.1 Data

The provided data in `NN_I_regr_data.Rda` and `NN_I_class_data.Rda` contain simulated data sets that can be loaded using the function `load()`.

The R objects provided in both file are named identically as follows:

- `x1`, `x2`: two input variables (training data)
- `y`: the corresponding outcome variable
- `x1val`, `x2val`, `yval`: an equivalent validation data set

The files `ML2_11_NN_I_1.R`, `ML2_11_NN_I_2.R` and `ML2_11_NN_I_3.R` contain the code to work on the corresponding exercises (see below).

2.2 Tips

Useful to re run the NN with a different number of iterations.

- A user defined function in your script file, needs to be loaded into R after every time it is modified. To do this put the cursor somewhere in the function and press *Ctrl + Alt + f*.
- To run a whole block of code, that is defined using `{ }`, put the cursor on the same line as `{` or `}` and press *Ctrl +*.



2.3 Functions

In each exercise the function `NN`, `NN2` and `NN3` run the neural network for a given set of weights and biases. These parameter values and the observation values are passed into the function as arguments.

The functions make use of the R vector arithmetic to fit the current set of parameters to the all the observations in one sweep. Take, for example, the line in the function `NN`

```
z1 <- whl11 * x1 + whl12 * x2 + bh11
```

`x1` and `x2` are vectors, so the resulting object `z1` is also a vector. This is much faster than using a `for` loop on each observation.

2.4 Optimization function

The **minimisation routine** used in the code is a rejection routine. It tries out a new parameter value, if the loss function is lower then that new parameter value is accepted but is rejected if it is worse.

An outline of the routine is:

1. Let θ^{cur} be the current vector of parameters with length Q .
2. Simulate Δ , *one* normally distributed random number with standard deviation `window`.
3. Perturb only the q -th element of θ^{cur} by this random amount and set

$$\theta_s^{\text{new}} = \begin{cases} \theta_s^{\text{cur}} + \Delta & \text{for } s = q \\ \theta_s^{\text{cur}} & \text{for } s \neq q \end{cases}$$

4. Run the NN using this new parameter vector θ^{new} and calculate the loss function.
5. If the new loss function is smaller than before, update $\theta^{\text{cur}} = \theta^{\text{new}}$ otherwise reject this attempt.
6. Loop q over each element $1, \dots, Q$ and repeat ad infinitum!

This minimisation algorithm is very easy to code but it is slow and requires many thousands of iterations.



3 Exercises

3.1 One hidden layer with 1 node

This exercise is based on the R script `ML2_11_NN_I_1.R`.

We will start with a neural network with one hidden layer and just one node in the hidden layer. The parameters are

- hidden layer:
 - `wh111`: w coefficient in hidden layer node 1 for x_1
 - `wh112`: w coefficient in hidden layer node 1 for x_2
 - `bh11`: bias coefficient in hidden layer node 1
- activation layer
 - `wo11`: w coefficient in output layer for activation a_1
 - `bo1`: bias coefficient in output layer

Work through the code that consists of the following steps. Whenever there are missing code elements indicated as `???` you have to complete the code.

- 1) Load the data
- 2) Scale the training and validation data.
- 3) Define the sigmoid function and examine the given function `NN`, so that you understand how it works.
- 4) The initial number of iterations is 10, so each parameter will be updated at most twice. Run the for loop. Look at the “best so far” SSE and plot the current predicted values against the true values.
- 5) Increase the number of iterations to 10, 1000, 10^4 and 10^5 and after each run write down the training and validation SSE.
- 6) There is a commented out command which slowly reduces the window. Uncomment this line of code. Reducing the window with the number of iterations size seems to reduce the loss a little bit quicker.
- 7) Re-run the code several times notice that the validation MSE value varies more than the training MSE.

3.2 One hidden layer with 2 nodes

This exercise is based on the R script `ML2_11_NN_I_2.R`. The code is very similar to the code you worked on in the previous exercise 3.1.

We extend the parameter set with the following *additional* parameters:

- hidden layer:
 - `wh121`: w coefficient in hidden layer node 1 for x_1
 - `wh122`: w coefficient in hidden layer node 1 for x_2
 - `bh12`: bias coefficient in hidden layer node 1
- activation layer:
 - `wo12`: w coefficient in output layer for activation a_1

Work through the code. Whenever there are missing code elements indicated as `???` you have to complete the code.

- 1) Complete the function named `NN2` so that it reflects a NN with one hidden layers and two nodes.



- 2) Adjust the `param` vectors.
- 3) Repeat all steps of exercise 3.1 for the new neural network.
- 4) Compare your results to the results in exercise 3.1.

3.3 Neural network as a classifier: one hidden layer with 2 nodes, output layer with 3 nodes

This exercise is based on the R script `ML2_11_NN_I_3.R` and data set `NN_I_class_data.Rda`. The code is again very similar to the code you know from the previous exercises.

For classification the output layer requires as many nodes as there are classes. the data `y` has $K=3$ classes (labelled 1, 2 and 3).

Therefore the function `NN2` has to be extended so that 3 output activations and the final sigmoid function are considered. This means that the following *additional* parameters for the expanded output layer will be considered:

- hidden layer: unchanged
- output layer:
 - `w0111`: w coefficient in output layer for activation a_1 from hidden layer node 1
 - `w0121`: w coefficient in output layer for activation a_2 from hidden layer node 1
 - `w0131`: w coefficient in output layer for activation a_3 from hidden layer node 1
 - `w0112`: w coefficient in output layer for activation a_1 from hidden layer node 2
 - `w0122`: w coefficient in output layer for activation a_2 from hidden layer node 2
 - `w0132`: w coefficient in output layer for activation a_3 from hidden layer node 2
 - `b011`: bias in output layer for activation a_1
 - `b012`: bias in output layer for activation a_2
 - `b013`: bias in output layer for activation a_3

The function `NN3` has 3 output activations (don't forget to apply the sigmoid function). The activations then need to be normalised to give a predicted probability for each class. This last step uses a couple of *R Tricks* to be able to do this without using a for loop, so this step has been provided for you.

Preparations:

- Save the code developed in exercise 3.1 to a new file and use it as template to make the following adjustments.

Tasks:

- 1) Scale the data sets `x1`, `x2`, `x1val`, `x2val` used for classification.
What about scaling `y` and `yval`?
- 2) Complete the function named `NN3` so that it reflects a NN with one hidden layers and two nodes as well as an output layer with three nodes.
- 3) Adjust the `param` vectors in the given code.
- 4) Adjust the model fit criteria to cross entropy as discussed in the lecture.
- 5) Adjust the visualisation to a dot plot to check the model fit and use a classification matrix to judge the model fit.
- 6) Now, repeat all steps of the previous exercises for the classifier neural network.



3.4 One-dimensional optimisation

The function minimised we consider here is a polynomial of degree

$$f(x) = 3x^4 + 5x^3 - 20x^2 + 8x + 10.$$

The code is given in the file `ML2_11_optim_1d.R`.

- a) Work through this example yourself.
- b) Define a new function called `sinf` which corresponds to $f(x) = \sin(x)$.
 - Copy the 1-dim minimisation code and adapt it for this problem.
 - You will need to define $f'(x)$
- c) Try different starting values to see how this affects the found minimum.
- d) Try adapting the step length `s` so that the convergence is quicker. For example try increasing the step size slightly with iteration number.
- e) Use $x = 7.8$ as the starting point.
 - Run a couple of iterations one by one inspecting the value of x after each iteration.
 - Then run 10 iterations in one go.
 - What do you notice about the values of x ?
 - Can you explain why?
 - Continue until you have reached the local minimum.
- f) Try finding the local minimum for $f(x) = e^{x^2+2x-4}$.
 - You will need to use the chain rule to calculate the derivative.
 - What difficulties do you observe?

3.5 Two-dimensional optimisation

The code for the second demonstration is given in `ML2_11_optim_2d.R`

- a) Work through the demo code for

$$f(x_1, x_2) = x_1^4 x_2^2 + (x_1 - 1)^2 (x_2 - 1)^4$$

trying different starting points.

- b) The outline code has been given for this part. The function to minimise is

$$f(x) = x_1^2 + x_2^2 + 2x_1 - 4x_2 - 1.$$

Calculate the partial derivatives and complete the code to find the approximate minimum.

- c) Use calculus to obtain the exact location of the function minimum.



3.6 Linear model parameter optimisation

The code for this exercise contained in `ML2_11_optim_ols.R` defines a quadratic regression model of the form

$$y_i = \beta_1 + \beta_2 x_i + \beta_3 x_i^2 + e_i$$

where e_i is the error term with a $N(0, 0.2^2)$ distribution.

The data consisting of $n=25$ observations are simulated in the code.

Usually the parameters β_1 , β_2 , and β_3 are calculated using exact solutions to the so called normal equations, giving the least squares estimates. In this exercise you will obtain the results using Newton-Raphson minimisation.

The loss function for a linear model is the residual sum of squares

$$L(\beta) = \sum_{i=1}^{25} (y_i - \beta_1 - \beta_2 x_i - \beta_3 x_i^2)^2$$

- Derive the partial derivatives regarding $\theta^T = (\beta_1, \beta_2, \beta_3)$.
- Complete the code and confirm that the parameters converge to values close to those specified in the simulation code.
- Check your outcome by running a OLS estimation for θ .



4 Written Homework exercises

4.1 Newton-Raphson method: finding a root of a function

A function can be approximated around a given point x_0 using the first order Taylor series

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0).$$

This is a linear polynomial, which uses the function, and the derivative value evaluated at the point x_0

- a) Rearrange the expression so that it is in the form

$$x \approx \quad (1)$$

If x_1 is a root of $f(x)$, then $f(x_1) = 0$. Using this and Equation (1), show that x_1 approximately satisfies the following equation: $x_1 \approx x_0 - \frac{f(x_0)}{f'(x_0)}$. x_1 will not be an exact root, because of the approximation, but multiple iterations of the following formula usually give a better and better approximation for a root of $f(x)$.

This leads directly to the Newton-Raphson method for finding a root of f :

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

- b) Let $f(x) = 3x^2 - 4x - 5$ and $x_0 = -2$. Carry out 2 iterations of the N-R method to obtain x_2 .

4.2 Finding a minimum of a function

A very similar method can be used to minimise a function using the second order Taylor series.

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2$$

This is a quadratic polynomial, which uses the function, first and second derivative value evaluated at the point x_0

- a) Differentiate both sides of the above equation to obtain

$$f'(x) \approx \quad (2)$$

- b) Use an analogous argument to the above, to obtain an iterative procedure that finds a point where $f'(x_i) = 0$
- c) Show that when $f(x) = 3x^2 - 4x - 5$ and $x_0 = -2$, the true minimum is found after just one iteration. The reason for this is that f is quadratic and the second order Taylor series is exact for quadratic functions.



4.3 Chain rule in back propagation

- a) Using the NN from the lecture, obtain the following partial derivatives

$$\frac{\partial R}{w_{12}^{(2)}} \quad \text{and} \quad \frac{\partial R}{b_1^{(2)}}$$

for a NN with one output node used for regression.

- b) Considering the sigmoid function $\sigma(v) = (1 + e^{-v})^{-1}$.
- Calculate $\sigma'(v)$ and
 - show that $\sigma'(v) = \sigma(v)(1 + \sigma(v))$.